

II. РАБОТА С КОНТРОЛИ И ИЗПОЛЗВАНЕ НА МЕТРИКИ В СОФТУЕРНОТО ПРОИЗВОДСТВО

Цел на упражнението

Това упражнение запознава студентите с често използвани контроли в C# Windows Forms приложения, с достъп до техните свойства, методи, събития, с автоматично подреждане на контроли. Представя примери с управляващи оператори и функции, както и имплементация на основни метрики в софтуерното производство.

1.Контроли

2.Свойства

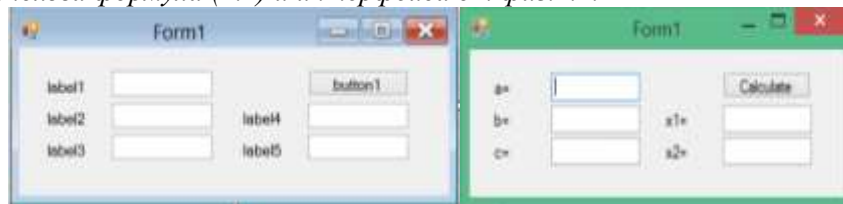
3.Събития (events)

4.Обработка на събития

5.Автоматично подреждане на контроли

6.Задачи за изпълнение

Задача 1. Да се създаде приложение Квадратно уравнение от вида $a \cdot x^2 + b \cdot x + c = 0$. За целта да се използва формула (2.1) и интерфейса от фиг.2.1.



Фиг.2.1. Интерфейс на приложение Квадратно уравнение, в режим проектиране и изпълнение

Да се пресметнат дискриминантата и квадратните корени на уравнението по зададени стойности за a , b и c . Да се направи проверка за коректни входни данни, да се изведат подходящи съобщения, при два еднакви корена или при липса на реални корени. Резултатът да се закръгли до втори знак след десетичната запетая. Да се направят подходящи промени в свойствата на контролите извеждащи корените (дискриминантата), така, че потребителят да не може да ги променя ръчно. Задачата да се реализира по два начина с и без бутон.

$$x_{1/2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (2.1)$$

Примерно решение – Вариант 1:

```
private void Form1_Load(object sender, EventArgs e) {
    label1.Text = "a="; label2.Text = "b=";
    label3.Text = "c="; label4.Text = "x1=";
    label5.Text = "x2="; button1.Text = "Calculate"; }
private void button1_Click(object sender, EventArgs e){
    double a, b, c, d, x1, x2;
    try{
        a = double.Parse(textBox1.Text);
        b = double.Parse(textBox2.Text);
        c = double.Parse(textBox3.Text);
        d = Math.Pow(b, 2) - 4 * a * c;
        if (d >= 0) {
            x1 = (-b + Math.Sqrt(d)) / (2 * a);
            x2 = (-b - Math.Sqrt(d)) / (2 * a);
            textBox4.Text = x1.ToString();
            textBox5.Text = x2.ToString();}
        else MessageBox.Show("Error!!!");}
    catch { MessageBox.Show("Error2!!!"); }}
```

Примерно решение – Вариант 2:

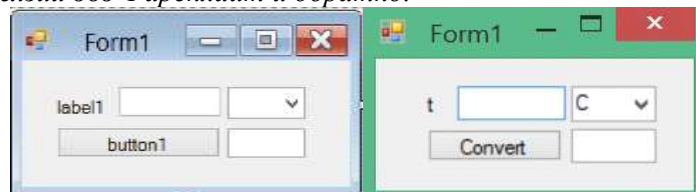
```
public static class Discriminant{
    public static double dd(double a, double b, double c) {
        return Math.Sqrt(Math.Pow(b, 2) - 4 * a * c); }
    public static double x1(double a, double b, double c) {
        return (-b + dd(a, b, c)) / (2 * a);}
```

```

    public static double x2(double a, double b, double c) {
        return (-b - dd(a, b, c)) / (2 * a); } }
namespace UP2_1V2{
    void clear(){
        textBox1.Clear(); textBox2.Clear(); textBox3.Clear();
        textBox4.Clear(); textBox5.Clear(); textBox6.Clear();}
    private void button1_Click(object sender, EventArgs e){
        clear();}
    private void Quadratic_equation_Load(object sender, EventArgs e){
        label1.Text = "a=";          label2.Text = "b=";
        label3.Text = "c=";          label4.Text = "D=";
        label5.Text = "x1=";         label6.Text = "x2=";
        button1.Text = "Clear";       button2.Text = "Calculate";
        button3.Text = "To the main"; textBox4.Enabled = false;
        textBox5.Enabled = false;     textBox6.Enabled = false; }
    private void button3_Click(object sender, EventArgs e){
        this.Close(); }
    private void button2_Click(object sender, EventArgs e) {
        double a, b, c, d, x1, x2;
    try {
        a = Convert.ToDouble(textBox1.Text);
        b = Convert.ToDouble(textBox2.Text);
        c = Convert.ToDouble(textBox3.Text);
        d = Discriminant.dd(a, b, c);
        if (d >= 0){
            x1 = Discriminant.x1(a, b, c);
            x2 = Discriminant.x2(a, b, c);
            textBox4.Text = d.ToString();
            textBox5.Text = String.Format("{0:0.##}", x1);
            textBox6.Text = String.Format("{0:0.##}", x2); }
        else {MessageBox.Show("Impossible division by zero!"); clear();}
    } catch {
        MessageBox.Show("Incorrect data!");
    }
    clear(); } }

```

Задача 2. Да се разработи приложение Температурен конвертор за преобразуване на температура от Целзий във Фаренхайт и обратно.



Фиг.2.2. Интерфейс на приложение Температурен конвертор, в режим проектиране и изпълнение

Да се направят подходящи промени в свойствата на контролата за извеждане на резултата и тази за избор на мерната единица за температурата, така, че да не може да се променят те ръчно. Да се закръгли резултатът до втори знак след десетичната запетая. Задачата да се реализира по два начина с и без бутон. За целта да се приложат формула (2.2) и (2.3) и се използва интерфейс от фиг.2.2.

$$F = (C * 9) / 5 + 32 \quad (2.2)$$

$$C = (F - 32) * 5 / 9 \quad (2.3)$$

Примерно решение:

```

void ConvertTemperature(){
    double Far, Cel;
    if (textBox5.Text != ""){
        if (comboBox2.Text == "C") {
            Cel = double.Parse(textBox5.Text);
            Far = (Cel * 9) / 5 + 32;
            textBox6.Text = Far.ToString(); }
        else {

```

```

Far = double.Parse(textBox5.Text);
Cel = (Far - 32) * 5.00 / 9.00;
textBox6.Text = Cel.ToString(); } } }

```

Методът ConvertTemperature(); се извиква на 3 места в методите: textBox5_TextChanged, comboBox2_SelectedIndexChanged, както и textBox6_TextChanged.

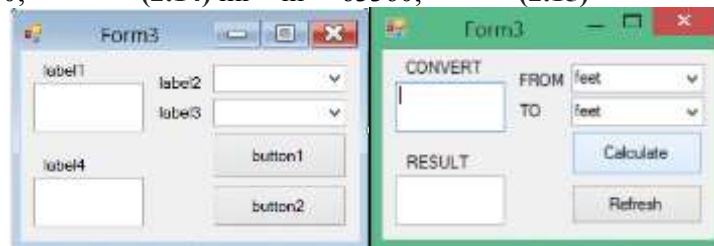
Задача 3. Да се разработи приложение Конвертор за разстояния, като се използват мерните единици: feet, yard, inch и mile. Конверторът да преобразува разстоянието след избор от една в друга мерна единица. Да се направят подходящи промени в свойствата на контролата за извеждане на резултата и тези за избор на мерна единица за разстояние, така, че да не могат да се променят ръчно. Задачата да се реализира по два начина с и без бутон. За целта да се използват Таблица 2.1. и интерфейс от фиг.2.3.

Таблица 2.1.

	feet	yards	inches	miles
feet	1	0.33333	12	0.0001894
yards	3.000	1	36	0.0005682
inches	0.0833333	0.0277778	1	1.57828E-5
miles	5280	1760	63360	1

Формули за преобразуване:

$ft = in * 12.000;$ (2.4) $ft = yd * 3.000;$ (2.5)
 $ft = mi * 0.0001893939;$ (2.6) $yd = ft * 0.33333;$ (2.7)
 $yd = mi * 0.0005681818;$ (2.8) $yd = in * 36;$ (2.9)
 $in = ft * 0.08333333333;$ (2.10) $in = yd * 0.0277777778;$ (2.11)
 $in = mi * 1.57828E-5;$ (2.12) $mi = ft * 5280;$ (2.13)
 $mi = yd * 1760;$ (2.14) $mi = in * 63360;$ (2.15)



Фиг.2.3. Интерфейс на приложение Конвертор за разстояния, в режим проектиране и изпълнение

Частично решение (без бутон):

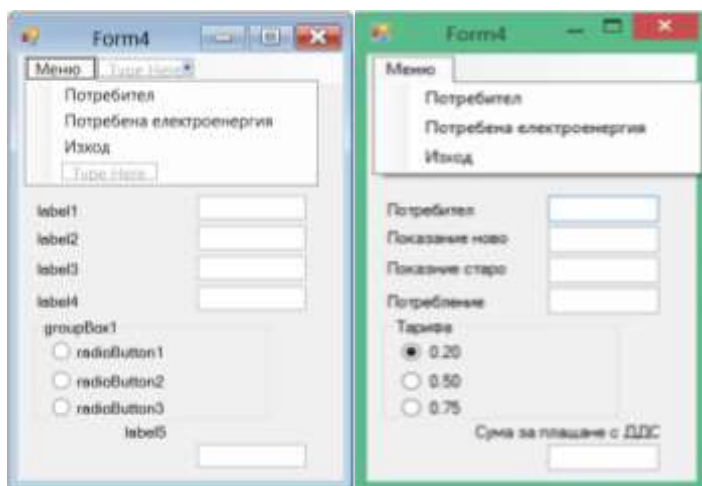
```

double[,] matrix;
private void Form1_Load(object sender, EventArgs e) {
    comboBox1.Items.Add("feet");
    ... comboBox1.SelectedIndex = 0;
    comboBox2.Items.Add("feet");
    ... matrix= new double[,]
    {{1, 0.33333, 12, 0.0001894},
    {3.000, 1, 36, 0.0005682},
    {0.0833333, 0.0277778, 1, 1.57828E-5},
    {5280, 1760, 63360, 1} }; }
void CalculateConvert(){
    if (textBox1.Text!="")
        textBox2.Text = (double.Parse(textBox1.Text) * (matrix[comboBox1.SelectedIndex,
        comboBox2.SelectedIndex])). ToString();}

```

Методът CalculateConvert(); се извиква на три места: textBox1_TextChanged, comboBox1_SelectedIndexChanged, както и comboBox2_SelectedIndexChanged.

Задача 4. Да се разработи приложение Потребление на електроенергия, което да изчислява консумираната енергия и сума за плащане от потребител, след въведени ново и старо показание на електромер и избрана тарифа. Приложението да разполага с меню, което включва: потребител, потребена енергия и изход от програмата.

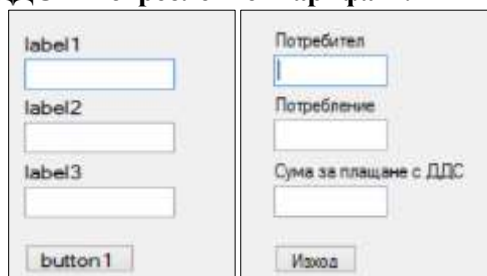


Фиг.2.4. Интерфейс на приложение Потребление на електроенергия, формуляр Потребител

Да се приложат формули (2.16) и (2.17) за изчисляване и да се използват интерфейсите от фиг.2.4 и фиг.2.5. за формуляр Потребител и Потребена електроенергия.

$$\text{Потребление} = \text{Старо показание} - \text{Ново показание} \quad (2.16)$$

$$\text{Сума с ДДС} = \text{Потребление} * \text{Тарифа} * 1.2 \quad (2.17)$$



Фиг.2.5. Интерфейс на приложение Потребление на електроенергия, формуляр Потребена електроенергия

Приложението да разполага с управляващо меню и формуляр за клиент с полета за въвеждане на старо и ново показание, поле потребление (изчислява се автоматично), три опции за избор на тарифа на различна стойност в лева. На базата на направения избор автоматично да се изчислява и сумата за плащане с ДДС. При избор на потребител поредният номер на клиент да нараства с единица. При избор на потребена енергия да се изведат данните за потребление на текущия потребител в нов екран (формуляр) с общи данни: номер на потребител, консумирана енергия и сума за плащане (фиг.2.5). При избор изход да се изведе съобщение за съгласие за напускане на приложението (при Yes) или прехвърляне към формуляр за потребител (при No). След всяка промяна в избора на тарифа, автоматично да се преизчислява сумата за плащане.

Задача 5. Да се създаде приложение Лице на фигура. Да се напише функция за изчисляване лице на правоъгълник по зададени две страни a и b , чиито стойности се получават при ръчно отместване на плъзгача на контролите $HScrollBar$ и $VScrollBar$. При отместване на плъзгача по хоризонтала на $HScrollBar$ се променя параметър a , а по вертикала на $VScrollBar$ се променя параметър b . На базата на отместването по хоризонтала и вертикала да се изчисли автоматично лицето на фигурата. За целта да се приложи формулата за намиране лице на правоъгълник.

7. Задачи за самостоятелна работа

Задача 1. Да се напише приложение Бройна система за преобразуване на числа от една в друга бройна система (двупосочно от десетична в двоична, осмична и шестнадесетична бройна система и обратно). Да се извърши проверка за валидни входни данни.

Задача 2. Да се създаде приложение Логаритмично уравнение от вида:

$$X = \log_a b \quad (2.18)$$

При задаване на произволни две променливи от уравнението да се изчисли третата. Да се извърши проверка за коректни входни данни.